

## AI Report

We are moderately confident this text is

# AI Generated

### AI Probability

# 69%

This number is the probability that the document is AI generated, not a percentage of AI text in the document.

### Plagiarism



The plagiarism scan was not run for this document. Go to [gptzero.me](https://gptzero.me) to check for plagiarism.

## LAPORAN TUGAS 1 - 4 Yoanre Louis A.H.P (2410651039).pdf - 10/19/2025

LAPORAN PRAKTIKUM TUGAS 1 - 4 SISTEM OPERASI Dosen Pengampu: Triawan Adi Cahyanto, M.Kom Disusun Oleh: Yoanre Louis A.H.P (2410651039) PRODI TEKNIK INFORMATIKA FAKULTAS TEKNIK UNIVERSITAS MUHAMMADIYAH JEMBER 2025 BAB 1 PENDAHULUAN 1.1 LATAR BELAKANG Sistem operasi merupakan fondasi utama dalam pengoperasian perangkat komputasi modern, yang berperan sebagai mediator antara perangkat keras (hardware) dan perangkat lunak (software). Pemahaman mendalam mengenai konsep-konsep inti sistem operasi seperti manajemen proses, komunikasi antar proses (IPC), manajemen memori, dan sistem file menjadi kompetensi esensial bagi calon ahli teknologi informasi. Praktikum ini dirancang untuk menjembatani teori dan praktik melalui implementasi langsung fitur-fitur sistem operasi menggunakan system calls Linux. Pengembangan Program Process Manager, File Monitor, Custom Shell, dan System Information Tool memungkinkan eksplorasi terhadap mekanisme internal seperti pembuatan proses paralel dengan fork(), komunikasi data melalui pipe() dan shared memory, monitoring real-time filesystem dengan inotify, serta ekstraksi informasi sistem melalui virtual filesystem /proc. Pengalaman hands-on ini tidak hanya memperkuat pemahaman konseptual tetapi juga mengasah kemampuan dalam mengembangkan aplikasi sistem yang efisien dan robust, mempersiapkan mahasiswa untuk menghadapi tantangan optimasi kinerja dan pemecahan masalah dalam lingkungan komputasi modern yang kompleks. 1.2 RUMUSAN MASALAH 1. Bagaimana mengimplementasikan manajemen proses dan komunikasi antar proses menggunakan system calls fork(), pipe(), dan shared memory? 2. Bagaimana membangun sistem monitoring file yang dapat mendeteksi perubahan secara real-time menggunakan inotify dan menangani signal SIGTERM? 3. Bagaimana mengembangkan custom shell dengan fitur pipes, redirection, background processes, serta tool monitoring sistem yang membaca informasi dari /proc? 1.3 TUJUAN 1. Memahami dan mengimplementasikan konsep multiprocessing dan IPC dalam program Process Manager. 2. Membangun sistem file monitoring yang robust dengan kemampuan real-time detection dan graceful termination. 3. Mengembangkan custom shell dengan fitur advanced dan system information tool yang mampu mengekstrak data hardware/software secara langsung dari kernel Linux. BAB II PEMBAHASAN 2.1 PRAKTIK TUGAS 1 2.1.1 Program Process Manager Program Process Manager ini mengimplementasikan konsep multiprocessing dengan fork() untuk membuat tiga child process yang berjalan paralel. Setiap child process menjalankan tugas komputasi berbeda: perhitungan faktorial, pencarian bilangan prima, dan pembalikan string. Program menggunakan pipe untuk IPC komunikasi hasil komputasi child1 dan child2 ke parent process, serta memanfaatkan shared memory untuk child3 sesuai re-

quirement NIM ganjil. Parent process menggunakan `waitpid()` untuk sinkronisasi dan `WIFEXITED/WEXITSTATUS` untuk tracking status termination. Meskipun muncul warning implicit return type selama kompilasi, program tetap berhasil dieksekusi dengan demonstrasi konkurensi process dan mekanisme IPC yang efektif.

### 2.1.2 Penjelasan Proses Kompilasi dan Eksekusi:

1. `cd praktikum_os`: User berpindah ke direktori `praktikum_os` untuk mengakses file program
2. `nano process_manager.c`: Membuka file source code C menggunakan editor nano untuk melihat atau mengedit kode
3. `gcc process_manager process_manager.c`: Melakukan kompilasi file C menjadi executable, tapi dengan syntax yang salah  
Warning 1: return type defaults to 'int' - fungsi `hitung_faktorial` tidak dideklarasikan tipe return-nya  
Warning 2: 'sprintf' writing past the end - potensi buffer overflow karena `buffer[10]` terlalu kecil untuk angka 1234567890  
Error linking: invalid version 3 - gagal membuat executable karena syntax compile salah
4. `./process_manager`: Berhasil dijalankan meskipun sebelumnya gagal compile  
Program membuat 3 child process (PID 434, 435, 436) untuk tugas berbeda  
Menggunakan IPC: pipe untuk child1 & child2, shared memory untuk child3  
Parent process menggunakan `waitpid()` untuk sinkronisasi dan `WIFEXITED/WEXITSTATUS` untuk tracking status termination  
Output sukses: faktorial 5=120, 10 bilangan prima, string "ollah" dibalik jadi "hallo"  
Waktu eksekusi tercatat untuk setiap process

### 2.2 PRAKTIK TUGAS 2

#### 2.2.1 File Monitor Program

File Monitor ini menggunakan `inotify_init()` dan `inotify_add_watch()` untuk memantau direktori `/monitor_dir` dengan event `IN_CREATE`, `IN_MODIFY`, dan `IN_DELETE`. Setiap event dicatat ke `file_monitor.log` menggunakan `open()` dan `write()` dengan format [Timestamp] [Event Type] [File Name] [PID]. Untuk NIM ganjil (akhir 9), program menampilkan notifikasi email dummy pada setiap event. Program berjalan infinite loop hingga menerima `SIGTERM`, dimana signal handler melakukan cleanup dengan menutup file descriptor `inotify` dan log file secara proper.

#### 2.2.2 Penjelasan Proses Kompilasi dan Eksekusi Before (sebelum):

1. Persiapan Direktori: Membuat direktori `file_monitor_task` dan subdirektori `monitor_dir`. Direktori `monitor` dikondisikan dalam keadaan kosong
2. Kompilasi Program: File source `file_monitor.c` dikompilasi dengan GCC menghasilkan executable `file_monitor`
3. Inisialisasi Monitoring Program: Program dijalankan dengan target `/monitor_dir`. PID 1036 terassign untuk process monitor. File log `file_monitor.log` disiapkan untuk recording
4. Kesiapan Sistem: Program aktif dan siap mendeteksi perubahan. Mekanisme `inotify` initialized. Signal handler `SIGTERM` terkonfigurasi untuk graceful termination
5. Verifikasi Kondisi Awal: Terminal paralel mengonfirmasi direktori `monitor` kosong. Membuktikan kondisi sebelum testing dimulai. Struktur direktori sesuai untuk monitoring
6. Fungsi Utama: Real-time detection events: `CREATE`, `MODIFY`, `DELETE`. Logging otomatis ke file dengan timestamp. Notifikasi email dummy untuk NIM ganjil.

#### Testing (uji coba):

1. Terminal 1 (Program Monitoring): Program berjalan dengan PID 1079. Memantau direktori `/monitor_dir`. Mendeteksi 3 jenis event: `CREATED`, `MODIFIED`, `DELETED`. Notifikasi email dummy aktif untuk setiap event. Timestamp tercatat otomatis: 2025-10-16 22:33:40 dll. Log tersimpan di `file_monitor.log`
2. Terminal 2 (Aksi Testing):  
`touch test1.txt` Trigger event `CREATED` echo "hello"  
`>> test1.txt` Trigger event `MODIFIED`  
`touch file2.log` Trigger event `CREATED`  
kedua `rm test1.txt` Trigger event `DELETED`
3. Event Yang Terdeteksi: `CREATED - test1.txt (22:33:40)` `MODIFIED - test1.txt (22:30:01)` `CREATED - file2.log (22:30:21)` `DELETED - test1.txt (22:30:07)`
4. Fitur Yang Berhasil: Real-time file monitoring. Deteksi 3 jenis event filesystem. Notifikasi email dummy (NIM ganjil). Automatic logging dengan timestamp. Signal handling untuk graceful termination. Background process monitoring dengan PID.

#### After (Setelah)

1. Proses Termination: Program dihentikan menggunakan `kill 1079 (SIGTERM)`. Signal handler berhasil menangani `SIGTERM`. Pesan "Menerima signal `SIGTERM`, melakukan cleanup..." ditampilkan. Cleanup resource berhasil: menutup file descriptor `inotify` dan log file. Program berhenti secara graceful dengan pesan "Cleanup selesai. Program berhenti."
2. Hasil File Log: File `file_monitor.log` berisi 4 event yang tercatat: `CREATED - test1.txt (22:33:46)` `MODIFIED - test1.txt (22:34:01)` `CREATED - file2.log (22:34:21)` `DELETED - test1.txt (22:34:47)`. Format log konsisten: [Timestamp] [Event Type] [File Name] [PID]
3. Kondisi Direktori Setelah Testing: Direktori `monitor_dir` hanya berisi `file2.log`. File `test1.txt` sudah terhapus (sesuai event `DELETE`). Struktur direktori menunjukkan hasil akhir testing: `file2.log` (masih ada - created) `test1.txt` (sudah dihapus - deleted)
4. Fitur Yang Terbukti Berfungsi: Signal handling `SIGTERM` untuk graceful termination. Cleanup resource otomatis. Logging persisten ke file. Konsistensi antara event dan kondisi aktual direktori. Monitoring real-time dari awal sampai termination.

### 2.3 PRAKTIKUM TUGAS 3

#### 2.3.1 Custom Shell

Custom Shell merupakan program command-line interface yang dikembangkan secara manual menggunakan system calls untuk memahami konsep internal shell. Program ini mendukung eksekusi command dengan multiple pipes, redirection file, background processes, dan built-in commands seperti cd, pwd, serta echo. Implementasi menggunakan parsing manual dengan strtok() dan management process melalui fork(), execvp(), serta waitpid() tanpa bergantung pada library high-level. Shell ini berhasil menangani berbagai skenario kompleks seperti command "ls " grep my " wc -l" dan membuktikan zero memory leaks melalui valgrind report, menunjukkan pemahaman mendalam tentang process management dan I/O operations di level sistem operasi.

### 2.3.2 Penjelasan Proses Kompilasi dan Eksekusi

1. Proses Kompilasi: File source myshell.c dikompilasi menggunakan GCC Option -g digunakan untuk debug info Valgrind Output executable: myshell Tidak ada error atau warning selama kompilasi
2. EKSEKUSI DAN TESTING: Program dijalankan dalam environment Valgrind Command pwd menampilkan working directory echo "Testing custom shell" berfungsi normal Multiple pipes ls " grep my " wc -l menghasilkan output 2 Redirection ls > output.txt & berjalan di background Built-in commands cd /tmp dan pwd bekerja sesuai ekspektasi Background process teridentifikasi dengan PID terpisah
3. Hasil Valgrind: Memory Usage: 2 allocations, 2 frees Memory Leaks: 0 bytes in 0 blocks Errors: 0 errors from 0 contexts Kesimpulan: Tidak ada memory leaks terdeteksi
4. Fitur Yang Terbukti: Multiple pipes (NIM ganjil) Background processes dengan & File redirection (>, >>) Built-in commands (cd, pwd, echo) Memory management yang optimal Zero memory leaks

## 2.4 PRAKTIKUM TUGAS 4

### 2.4.1 System Information Tool

System Information Tool adalah program monitoring sistem yang mengumpulkan informasi hardware dan software secara real-time melalui interface kernel Linux. Tool ini membaca virtual filesystem `/proc` untuk mendapatkan data CPU, memory, processes, dan network, serta menggunakan system call `statvfs()` untuk informasi disk usage. Program menampilkan informasi dalam format tabel ASCII yang terstruktur meli

● Sentences that are likely AI-generated.

## FAQs

---

### What is GPTZero?

GPTZero is the leading AI detector for checking whether a document was written by a large language model such as ChatGPT. GPTZero detects AI on sentence, paragraph, and document level. Our model was trained on a large, diverse corpus of human-written and AI-generated text with support for English, Spanish, French, German, and other languages. To date, GPTZero has served over 10 million users around the world, and works with over 100 organizations in education, hiring, publishing, legal, and more.

### When should I use GPTZero?

Our users have seen the use of AI-generated text proliferate into education, certification, hiring and recruitment, social writing platforms, disinformation, and beyond. We've created GPTZero as a tool to highlight the possible use of AI in writing text. In particular, we focus on classifying AI use in prose. Overall, our classifier is intended to be used to flag situations in which a conversation can be started (for example, between educators and students) to drive further inquiry and spread awareness of the risks of using AI in written work.

### Does GPTZero only detect ChatGPT outputs?

No, GPTZero works robustly across a range of AI language models, including but not limited to ChatGPT, GPT-5, GPT-4, GPT-3, Gemini, Claude, and AI services based on those models.

### What are the limitations of the classifier?

The nature of AI-generated content is changing constantly. As such, these results should not be used to punish students. We recommend educators to use our behind-the-scenes [Writing Reports](#) as part of a holistic assessment of student work. There always exist edge cases with both instances where AI is classified as human, and human is classified as AI. Instead, we recommend educators take approaches that give students the opportunity to demonstrate their understanding in a controlled environment and craft assignments that cannot be solved with AI. Our classifier is not trained to identify AI-generated text after it has been heavily modified after generation (although we estimate this is a minority of the uses for AI-generation at the moment). Currently, our classifier can sometimes flag other machine-generated or highly procedural text as AI-generated, and as such, should be used on more descriptive portions of text.

### I'm an educator who has found AI-generated text by my students. What do I do?

Firstly, at GPTZero, we don't believe that any AI detector is perfect. There always exist edge cases with both instances where AI is classified as human, and human is classified as AI. Nonetheless, we recommend that educators can do the following when they get a positive detection: Ask students to demonstrate their understanding in a controlled environment, whether that is through an in-person assessment, or through an editor that can track their edit history (for instance, using our [Writing Reports](#) through Google Docs). Check out our list of [several recommendations](#) on types of assignments that are difficult to solve with AI.

Ask the student if they can produce artifacts of their writing process, whether it is drafts, revision histories, or brainstorming notes. For example, if the editor they used to write the text has an edit history (such as Google Docs), and it was typed out with several edits over a reasonable period of time, it is likely the student work is authentic. You can use GPTZero's Writing Reports to replay the student's writing process, and view signals that indicate the authenticity of the work.

See if there is a history of AI-generated text in the student's work. We recommend looking for a long-term pattern of AI use, as opposed to a single instance, in order to determine whether the student is using AI.